HUDM 5001 Programming for Data Science
Fall 2026 Teachers College, Columbia University

Mock Midterm Exam

Practice for the in-class midterm on Tuesday, October 27
This mock covers the sessions up to SQLite. The real midterm covers everything we have learned before it.

About this mock test. This mock test is for you to get familiar with the upcoming midterm exam. No submission is needed for this mock test. The midterm will be similar to this test in overall design, but the number of questions may differ.

Name: _____ **UNI:** _____

Suggested time: 60–80 minutes.

Rules. To get the most out of this practice, try it under the same rules as the real exam:

1. Closed book. You may use **one** one-sided, letter-size sheet of notes (your "cheat sheet") and a calculator.
2. No computers, phones, tablets, smartwatches, earbuds, or AI tools. Put them away before you start.
3. You will **not** write any code. Every question asks you to read, trace, explain, or choose.
4. Part A: choose one answer for each question and mark it on the answer grid on page 2. Only the grid is scored.
5. Part B: write your answers in the space provided. You may cite line numbers (for example, "line 12") instead of copying code. Show your reasoning, because partial credit is given.
6. Unless a question says otherwise, assume that `import json, os, sqlite3`, `import numpy as np` and `import pandas as pd` have already been run, that each snippet runs by itself in a fresh Python 3 session, and that no warning messages appear.

Part	Points	Score
A. Multiple choice (25 questions × 2 points)	50	
B1. Reading comprehension: academic research	25	
Total	**75**	

Turn the page when you are ready to start.

Designed by Chenguang Pan (cp3280@tc.columbia.edu), refined by Claude.ai.

Part A: Answer grid

Mark **exactly one** letter for each question. Only this grid is scored for Part A. If you change an answer, erase the old mark completely.

1.	A	B	C	D	**14.**	A	B	C	D
2.	A	B	C	D	**15.**	A	B	C	D
3.	A	B	C	D	**16.**	A	B	C	D
4.	A	B	C	D	**17.**	A	B	C	D
5.	A	B	C	D	**18.**	A	B	C	D
6.	A	B	C	D	**19.**	A	B	C	D
7.	A	B	C	D	**20.**	A	B	C	D
8.	A	B	C	D	**21.**	A	B	C	D
9.	A	B	C	D	**22.**	A	B	C	D
10.	A	B	C	D	**23.**	A	B	C	D
11.	A	B	C	D	**24.**	A	B	C	D
12.	A	B	C	D	**25.**	A	B	C	D
13.	A	B	C	D					

Part A: Multiple choice

25 questions × 2 points = 50 points. Choose the single best answer for each question and mark it on the answer grid (page 2).

1. What does this code print?

```
x = 5
y = x
x = 10
print(x, y)
```

A. `5 5`
B. `5 10`
C. `10 5`
D. `10 10`

2. What does this code print?

```
print(18 / 4, 18 // 4, 18 % 4)
```

A. `4.5 4 2`
B. `4.5 5 2`
C. `4 4.5 2`
D. `4.5 4 0.5`

3. The Series `temps` is built below. Which expression returns the temperature for Wednesday (65) **by its position**?

```
temps = pd.Series([68, 71, 65, 74],
                  index=["Mon", "Tue", "Wed", "Thu"])
```

A. `temps.loc[2]`
B. `temps.iloc[3]`
C. `temps.iloc["Wed"]`
D. `temps.iloc[2]`

4. `students` has 5 rows and 3 columns. What does this code print?

```
students = pd.DataFrame({
    "name":  ["Maya", "Liam", "Noah", "Ava", "Zoe"],
    "major": ["Stats", "Learning", "Measurement", "Stats", "Learning"],
    "gpa":   [3.8, 3.5, 3.9, 3.6, 3.7],
})
print(students[["name", "gpa"]].shape, students["gpa"].shape)
```

A. `(2, 5) (5,)`
B. `(5, 2) (5,)`
C. `(5, 2) (5, 1)`
D. `(2, 5) (1, 5)`

5. What does this code print?

```
word = "data"
word.replace("a", "o")
print(word)
```

A. `doto`
B. `dota`
C. `data`
D. `TypeError`

6. What does this code print?

```
grades = {"Ann": 90, "Bob": 85}
print("Bob" in grades, 90 in grades, 85 in grades.values())
```

A. `True True True`
B. `True False True`
C. `True False False`
D. `False True True`

7. What does this code print?

```
s = "regression"
print(s[2:5], s[-3:])
```

A. `gre ion`
B. `gres ion`
C. `egr ion`
D. `gre regress`

8. What does this code print?

```
df = pd.DataFrame({"team": ["A", "B", "A", "B", "A"],
                   "pts":  [10, 7, 12, 9, 8]})
means = df.groupby("team")["pts"].mean()
print(means["A"], means["B"])
```

A. `9.2 9.2`
B. `30 16`
C. `3 2`
D. `10.0 8.0`

9. Each line below is run by itself. Which one raises a `TypeError`?

A. `f"Total: {90}"`
B. `"Total: " + 90`
C. `"Total: " * 2`
D. `"Total: " + str(90)`

10. What happens when this code runs?

```
df = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
df.rename(columns={"a": "alpha"})
print(df["alpha"])
```

A. It prints the column's values, because `rename` changed `df` in place.
B. `KeyError`, because `rename` can only rename row labels, not columns.
C. `KeyError`, because `rename` returned a new DataFrame that was never assigned, so `df` still has the column `a`.
D. `TypeError`, because `rename` needs a list of new names, not a dict.

11. What does this code print?

```
scores = pd.Series([80, 90, np.nan, 70])
print(scores.mean(), scores.fillna(0).mean())
```

A. `80.0 60.0`
B. `60.0 80.0`
C. `nan 60.0`
D. `60.0 60.0`

12. In the code below, `n` counts the rows with `pts` above 8. Which expression **always** gives the same number as `n`, for any DataFrame `df` that has a numeric column `pts`?

```
df = pd.DataFrame({"pts": [10, 7, 12, 9, 8]})
n = df[df["pts"] > 8].shape[0]
```

A. `df["pts"].sum() > 8`
B. `df["pts"] > 8`
C. `len(df["pts"] > 8)`
D. `(df["pts"] > 8).sum()`

13. After these lines run, which statement is true?

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
b.append(4)
```

Tip. Lists, sets, dictionaries, NumPy arrays, and pandas Series and DataFrames are all *mutable* objects. Writing `b = a` does not copy anything. It only gives the same object a second name. So when you change the object through `b` (for example with `b.append(...)`), the change also shows up through `a`. To get a separate copy, write `b = a.copy()`. Also remember that `x is y` asks whether two names refer to the very same object, while `x == y` only asks whether their values are equal.

A. `a` is still `[1, 2, 3]`, because `b = a` made a separate copy.
B. `a == c` is `True`, because both lists hold the same values.
C. `a is b` is `True` and `a == c` is `False`.
D. `a is c` is `True`, because both lists were built from the same numbers.

14. What does this code print?

```
df = pd.DataFrame({"x": [10, 20, 30, 40, 50]})
print(df.loc[1:3]["x"].sum(), df.iloc[1:3]["x"].sum())
```

A. `90 50`
B. `50 90`
C. `50 50`
D. `90 90`

15. What does this code print?

```
x = np.array([[2, 3, 1],
              [3, 1, 2]])
print(x[x > 1].sum(), (x > 1).sum())
```

A. `4 10`
B. `10 4`
C. `4 4`
D. `12 4`

16. What happens when this code runs?

```
df = pd.DataFrame({"age":    [19, 25, 31, 22],
                   "member": [True, False, True, True]})
print(df[df["age"] > 20 and df["member"]])
```

A. It prints the rows where `age` is over 20 and `member` is `True`.
B. `KeyError`, because `member` is not a valid column name.
C. `TypeError`, because a number cannot be compared with a Boolean column.
D. `ValueError`, because `and` cannot combine two Series; `&` with parentheses is needed.

17. What does this code print?

```
with open("log.txt", "w") as f:
    f.write("A")
with open("log.txt", "a") as f:
    f.write("B")
with open("log.txt", "w") as f:
    f.write("C")
with open("log.txt", "a") as f:
    f.write("D")
with open("log.txt", "r") as f:
    print(f.read())
```

Tip. Mode `"w"` (write) first erases everything that is already in the file, and then writes the new text. Mode `"a"` (append) keeps what is already in the file and adds the new text at the end. Either mode creates the file if it does not exist yet.

A. `CD`
B. `ABCD`
C. `ABD`
D. `D`

18. What does this code print?

```
orders = pd.DataFrame({"cid": [1, 2, 3, 4, 5],
                       "amt": [50, 20, 70, 40, 30]})
cust = pd.DataFrame({"cid":  [3, 4, 5, 6],
                     "city": ["NYC", "LA", "NYC", "SF"]})
merged = pd.merge(orders, cust, on="cid", how="outer")
print(merged.shape)
```

A. `(3, 3)`
B. `(5, 3)`
C. `(6, 3)`
D. `(9, 3)`

19. A SQLite table `scores` is shown below (`NULL` means missing), and the cursor `cur` is connected to its database. What does the code print?

```
student | score
--------+------
a       | 80
b       | NULL
c       | 90
d       | NULL
e       | 70
```

```
query = "SELECT COUNT(*), COUNT(score), AVG(score) FROM scores"
print(cur.execute(query).fetchall()[0])
```

A. `(5, 5, 80.0)`
B. `(5, 3, 80.0)`
C. `(3, 3, 80.0)`
D. `(5, 3, 48.0)`

20. What does this code print?

```
df = pd.DataFrame({"name":  ["Ann", "Bob", "Cara", "Dan"],
                   "score": [70, 95, 60, 85]})
top = df.sort_values("score", ascending=False)
print(top.iloc[0]["name"], top.loc[0]["name"], top.head(2)["score"].sum())
```

A. `Ann Ann 165`
B. `Cara Ann 130`
C. `Bob Bob 180`
D. `Bob Ann 180`

21. A SQLite table `visits` is shown below, and the cursor `cur` is connected to its database. What does the code print?

```
page | device  | secs
-----+---------+-----
home | mobile  | 30
home | desktop | 95
shop | mobile  | 60
shop | mobile  | 100
blog | mobile  | 20
blog | desktop | 40
home | mobile  | 50
shop | desktop | 10
```

```
query = """
SELECT page, COUNT(*) AS n, ROUND(AVG(secs), 1) AS avg_secs
FROM visits
WHERE device = 'mobile'
GROUP BY page
ORDER BY avg_secs DESC
LIMIT 1
"""
print(cur.execute(query).fetchall()[0])
```

A. `('shop', 2, 80.0)`
B. `('home', 3, 58.3)`
C. `('blog', 1, 20.0)`
D. `('shop', 2, 160)`

22. What does this code print?

```
sales = pd.DataFrame({"store":  ["N", "S", "N", "S", "N", "E"],
                      "amount": [120, 80, 100, 90, 140, 60]})
avg = sales.groupby("store")["amount"].mean().to_dict()
avg["W"] = 0
gap = avg["N"] - avg["S"]
print(len(avg), gap)
```

A. `3 35.0`
B. `4 35.0`
C. `4 190`
D. `4 1`

23. What does this code print?

```
df = pd.DataFrame({"id":    [1, 2, 3, 4, 5],
                   "score": [72, 58, 91, 65, 85]})
summary = {"n_pass": df[df["score"] >= 65].shape[0]}
with open("out.json", "w") as f:
    json.dump(summary, f)
with open("out.json", "r") as f:
    back = json.load(f)
back["n_pass"] = back["n_pass"] + 1
print(back["n_pass"], summary["n_pass"])
```

A. `4 3`
B. `4 4`
C. `5 4`
D. `5 5`

24. What does this code print?

```
trials = pd.DataFrame({"subject": ["s1", "s2", "s1", "s2"],
                       "rt":      [500, 900, 700, 400]})
conn = sqlite3.connect("lab.db")
trials.to_sql("trials", conn, index=False, if_exists="replace")
q = """SELECT subject, AVG(rt) AS m FROM trials
       GROUP BY subject ORDER BY subject"""
res = pd.read_sql_query(q, conn)
conn.close()
m = np.array(res["m"])
print(res.shape, (m > 620).sum(), res.loc[1, "subject"])
```

A. `(4, 2) 2 s2`
B. `(2, 2) 1 s1`
C. `(2, 2) 650.0 s2`
D. `(2, 2) 1 s2`

25. What does this code print?

```
orders = pd.DataFrame({"cid": [1, 2, 3, 4],
                       "amt": [50, 20, 70, 40]})
cust = pd.DataFrame({"cid":  [1, 2, 3, 5],
                     "city": ["NYC", "LA", "NYC", "SF"]})
m = pd.merge(orders, cust, on="cid", how="left")
m["city"] = m["city"].fillna("unknown")
by_city = m.groupby("city")["amt"].sum().to_dict()
print(by_city["NYC"], by_city["unknown"], len(by_city))
```

A. `120 40 4`
B. `120 40 3`
C. `2 1 3`
D. `KeyError: 'unknown'`

Part B: Reading comprehension

One script, 25 points. Read the background, study the data and the script, then answer the questions that follow. Each question says which version of the code it is about. You never need to write code. Describe fixes in words and cite line numbers.

B1. [25 pts] Academic research: a teacher-efficacy survey

Setting. Maya is a research assistant in a university lab that evaluates a coaching program for new teachers. Ten teachers took part in a small pilot: five received coaching and five did not. Each teacher filled out a survey on *classroom-management efficacy*, and one teacher accidentally submitted the form twice. The survey has four items, each answered from 1 (strongly disagree) to 5 (strongly agree). Items `q1`, `q2` and `q4` are worded positively. Item `q3` ("My class is often out of control") is worded negatively. Maya wrote the script `efficacy_pilot.py` (next page) to clean the data and compare the two groups. Her principal investigator (PI) gave her these rules:

R1. Remove duplicate submissions (identical rows) before any analysis.

R2. Reverse-code `q3` so that $1 \leftrightarrow 5$, $2 \leftrightarrow 4$, and 3 stays 3. After that, a higher number always means higher efficacy.

R3. A teacher's *scale score* is the mean of the four items (with `q3` reversed). A teacher who skipped any of the four items gets no scale score and is left out of the group means.

R4. Attach each teacher's school district from `schools.csv`. A teacher whose school is not listed in that file must stay in the data, with the district left blank.

R5. Compare the two group means and save a short summary as JSON for the PI.

The data. The two input files, exactly as pandas displays them after `pd.read_csv` (`NaN` means a blank cell in the file):

`survey_raw.csv`

	resp_id	group	school_id	q1	q2	q3	q4
0	101	coaching	S1	4	5.0	2	4.0
1	102	coaching	S1	5	4.0	1	5.0
2	103	coaching	S2	3	NaN	3	4.0
3	104	coaching	S2	4	4.0	2	3.0
4	104	coaching	S2	4	4.0	2	3.0
5	105	coaching	S3	4	3.0	3	3.0
6	201	comparison	S3	4	3.0	3	4.0
7	202	comparison	S2	2	3.0	4	NaN
8	203	comparison	S1	4	3.0	3	3.0
9	204	comparison	S2	3	4.0	3	3.0
10	205	comparison	S3	3	3.0	4	4.0

`schools.csv`

	school	district
0	S1	North
1	S2	South

Maya's script: `efficacy_pilot.py`

```python
# efficacy_pilot.py: clean the pilot survey and compare the two groups
import json
import os
import pandas as pd

raw = pd.read_csv("survey_raw.csv")
schools = pd.read_csv("schools.csv")
print("rows read:", raw.shape[0])

# Step 1: remove duplicate submissions (identical rows)
df = raw.drop_duplicates().copy()
print("rows after dedupe:", df.shape[0])

# Step 2: how many answers are missing for each item?
items = ["q1", "q2", "q3", "q4"]
print(df[items].isna().sum())

# Step 3: reverse-code q3 (negatively worded; the scale runs from 1 to 5)
df["q3_r"] = 5 - df["q3"]

# Step 4: scale score = mean of the four items, using the reversed q3
df["scale"] = (df["q1"] + df["q2"] + df["q3_r"] + df["q4"]) / 4

# Step 5: attach each teacher's district from the school lookup table
df = pd.merge(df, schools, on="school_id", how="left")
print("rows after merge:", df.shape[0])

# Step 6: compare the two groups on the scale score
summary = df.groupby("group")["scale"].agg(["count", "mean"]).round(2)
print(summary)

# Step 7: difference in group means (coaching minus comparison)
diff = summary.loc["coaching", "mean"] - summary.loc["comparison", "mean"]
print("difference:", round(diff, 2))

# Step 8: save the cleaned table and a short summary for the PI
df.to_csv("survey_clean.csv", index=False)
result = {
    "n_scored": int(summary["count"].sum()),
    "mean_coaching": float(summary.loc["coaching", "mean"]),
    "mean_comparison": float(summary.loc["comparison", "mean"]),
}
with open("summary.json", "w") as f:
    json.dump(result, f)
print("saved:", os.path.exists("summary.json"))
```

About the script. It contains exactly one bug and one PI's rule violation. The bug stops the script with an error message. The rule violation does not stop anything, but it makes the numbers wrong. Each question below says which version of the code it is about.

(a) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Explain what line 30 prints (it is `print(summary)`). You do not need to calculate any numbers. Just describe in words what it returns, for example what each row and each column stands for. You may also sketch the table if you think that is easier.

(b) **[4 pts]** *Code version: Original script (this answer does not depend on the bug or the rule violation).* In plain English, what do Steps 3 and 4 (lines 18–22) do, and why must Step 3 happen before Step 4?

(c) **[5 pts]** *Code version: Original script.* Maya runs the script. It prints its first three results and then stops with the message `KeyError: 'school_id'`. (i) Which line raises the error? (ii) Explain the cause, using the two data tables. (iii) Describe in words how to fix it.

(d) **[5 pts]** *Code version: Assume the error in (c) has been fixed, so the script runs to the end.* Even so, one line breaks one of the PI's rules, and Python does not give any error message. (i) Which line is it, and which rule does it break? (ii) Explain the mistake. (iii) Explain how to revise that line.

(e) **[3 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Suppose line 25 used `how="inner"` instead of `how="left"`. What would line 26 print? Write the number, and explain in a sentence or two how you got it. (Hint: which schools are listed in `schools.csv`?)

Line 26 would print `rows after merge:` _____

(f) **[4 pts]** *Code version: Assume the problems in (c) and (d) have both been fixed.* Rule R3 says that a teacher who skipped any of the four items gets no scale score and is left out of the group means. Does the script follow this rule? Answer Yes or No, then explain why. Name the line(s) responsible and point to evidence in the data tables.